
Algorithm Consistent Hash Sharding

initialization:

virtual_nodes $\leftarrow$ *number_of_virtual_nodes*

ring $\leftarrow$ { }

sorted_keys $\leftarrow$ []

function ADDNODE(*node_identifier*)

for $i \in \text{range}(\text{virtual_nodes})$ **do**

v_node $\leftarrow$ *node_identifier* + (*i*)

k $\leftarrow$ *hash_md5*(*data* = *v_node*) mod *MAX_NUMBER_16_BITS*

ring[*k*] $\leftarrow$ *node_identifier*

sorted_keys.append(*k*)

end for

sorted_keys.sort()

end function

function GETNODE(*shard_key*)

if not *ring* **then**

return None

end if

k $\leftarrow$ *hash_md5*(*data* = *shard_key*) mod *MAX_NUMBER_16_BITS*

index $\leftarrow$ *_find_next_position*(*sorted_array* = *sorted_keys*, *target* = *k*)

return *ring*[*sorted_keys*[*index*]]

end function

function HASH_MD5(*data*)

hash_digest_128_bits $\leftarrow$ *md5_hash_function*(*data*)

digest_16_bits_truncated $\leftarrow$ *hash_digest_128_bits*

return *int*(*digest_16_bits_truncated*)

end function

function _FIND_NEXT_POSITION(*sorted_array*, *target*)

low $\leftarrow$ 0

high $\leftarrow$ *len*(*sorted_array*) - 1

while $low \leq high$ **do**

mid $\leftarrow$ (*low* + *high*) $\div$ 2

if *sorted_array*[*mid*] < *target* **then**

low $\leftarrow$ *mid* + 1

else

high $\leftarrow$ *mid* - 1

end if

end while

return *low* **if** $low < \text{len}(\text{sorted_array})$ **else** 0

end function
